MarketSight

Stock Market Prediction System

Developer's Guide

Prepared by: Zane Tessmer

COSC 471 - Winter 2026 | Okanagan College

March 17, 2026

Table of Contents

Analysis Models

1. Use Case Diagram
2. Domain Model
3. Activity Diagrams

Design Models 4. Architecture 5. Class Diagram 6. Sequence Diagrams

Code 7. Project Source Code Description 8. Testing Code Description 9. Continuous Integration Report

Analysis Models

1. Use Case Diagram

```
                        +----------------------------------------+
                        |            MarketSight                 |
                        +----------------------------------------+
                        |                                        |
+--------+              |  [UC-01] Get Stock Prediction          |
|        |---------->|  [UC-02] View Stock Chart              |
|  User  |              |  [UC-03] Browse Stock List             |
|        |---------->|  [UC-04] Batch Predict All Stocks      |
+--------+              |  [UC-05] Manage Account Settings       |
    |                   |                                       |
    |                   +----------------------------------------+
    |
+---------+             +----------------------------------------+
|  Admin  |--------->|  [UC-06] Manage User Accounts          |
| (super) |             |    - Add User                          |
+---------+             |    - Edit User                         |
                        |    - Delete User                       |
                        +----------------------------------------+
                                     |
+---------+             +----------------------------------------+
|  Data   |--------->|  [UC-07] Explore Market Data           |
| Analyst |             |    - Browse Stocks                     |
+---------+             |    - Run Inference                     |
                        |    - Manage Datasets                   |
                        +----------------------------------------+
                                     |
+-----------+           +----------------------------------------+
| Airflow   |------->|  [UC-08] Automated Pipeline            |
| Scheduler |           |    - Seed Market Data                  |
+-----------+           |    - Retrain ML Model                  |
                        |    - Snapshot Datasets                 |
                        +----------------------------------------+
```

Actors:

- **User:** Any authenticated or anonymous user accessing the web dashboard

- **Admin:** Superuser with elevated privileges for user management
- **Data Analyst:** Technical user accessing the Streamlit interface
- **Airflow Scheduler:** Automated system actor triggering pipeline jobs

2. Domain Model

```
+------------------+        +-------------------+       +------------------+
|      Stock       |        |    DailyPrice     |       |   Prediction     |
+------------------+        +-------------------+       +------------------+
| symbol: str (PK) |1    * | id: int (PK)      |       | symbol: str      |
| name: str        |------>| stock_symbol: str  |       | current_price: $ |
| sector: str      |       | date: datetime     |       | predicted_price:$|
| industry: str    |       | open: float        |       | predicted_return:|
| exchange: str    |       | high: float        |       | confidence: 0-1  |
| ipo_date: date   |       | low: float         |       | model_version:str|
| market_cap: float|       | close: float       |       | prediction_date  |
+------------------+       | volume: int        |       +------------------+
                           +-------------------+
                                    |
                                    | feeds into
                                    v
                          +-------------------+
                          | FeatureEngineering|
                          +-------------------+
                          | 53 indicators:    |
                          | - RSI (14-period) |
                          | - MACD (12/26/9)  |
                          | - Bollinger Bands |
                          | - SMA/EMA (5-100) |
                          | - Lag features    |
                          | - Volume metrics  |
                          | - Volatility (ATR)|
                          +-------------------+
                                    |
                                    v
+------------------+       +-------------------+       +------------------+
|    XGBoost       |       |  ConfidenceCalc   |       |      User        |
|    Model         |       +-------------------+       +------------------+
+------------------+       | volatility: 35%   |       | id: int (PK)     |
| model.ubj        |       | return_mag: 30%   |       | email: str       |
| meta.json        |       | volume: 20%       |       | full_name: str   |
| ticker_encoder   |       | rsi: 15%          |       | hashed_password  |
| feature_names[]  |       | -> score: 0.0-1.0 |       | is_superuser:bool|
+------------------+       +-------------------+       | is_active: bool  |
                                                       +------------------+
```

Key Relationships:

- A **Stock** has many **DailyPrice** records (1:N)
- DailyPrice data feeds into the **FeatureEngineering** pipeline to produce 53 features
- The **XGBoost Model** consumes features and outputs a predicted return
- **ConfidenceCalc** evaluates market conditions to produce a reliability score
- **Users** interact with the system and are managed by **Admins**

3. Activity Diagrams

3.1 Prediction Flow

```
[User selects stock]
        |
        v
[Frontend sends GET /api/v1/inference/predict/{symbol}]
        |
        v
[Backend: Validate stock exists] --No--> [Return 404]
        |
       Yes
        v
[Backend: Fetch OHLC data (150+ bars)]
        |
        v
[Sufficient data?] --No--> [Return 400: Insufficient data]
        |
       Yes
        v
[Feature Engineering: Calculate 53 indicators]
        |
        v
[Encode ticker symbol]
        |
        v
[XGBoost model.predict(features)]
        |
        v
[Calculate predicted_price = current * (1 + predicted_return)]
        |
        v
[Calculate confidence score (volatility, RSI, volume, return)]
        |
        v
[Return PredictionResponse JSON]
        |
        v
[Frontend: Display prediction card with confidence color]
```

3.2 Data Pipeline Flow (Airflow)

```
[Airflow Scheduler triggers DAG]
        |
        v
[FMP API: Download OHLCV data (chunked)]
        |
        v
[Fill data gaps (2020-2023)]
        |
        v
[Load data into PostgreSQL]
        |
        v
[Feature Engineering: Calculate 53 indicators]
        |
        v
[Upload features to PostgreSQL]
        |
        v
[Extract training dataset]
        |
        v
[Optuna HPO: Optimize XGBoost hyperparameters]
        |
        v
[Train final model with best params]
        |
        v
[Export model.ubj + meta.json]
        |
        v
[Backend auto-loads new model artifact]
```

Design Models

4. Architecture

4.1 System Architecture Diagram

```
+-----------------------------------------------------------------------+
|                            Client Layer                               |
|  +------------------+    +------------------+    +------------------+ |
|  | React 19 + TS    |    | Streamlit App    |    | Swagger UI       | |
|  | Vite + Tailwind  |    | Data Explorer    |    | /docs, /redoc    | |
|  | TanStack Router  |    | Inference Test   |    |                  | |
|  +--------+---------+    +--------+---------+    +--------+---------+ |
+-----------|------------------------|----------------------|----------+
            |                        |                      |
            v                        v                      v
+-----------------------------------------------------------------------+
|                        Nginx Reverse Proxy (SSL)                      |
+------------------------------+----------------------------------------+
                               |
                               v
+-----------------------------------------------------------------------+
|                          Application Layer                            |
|  +-------------------------------------------------------------+  |
|  |                    FastAPI Backend (Python 3.12)              |  |
|  |  +----------+  +----------+  +----------+  +----------+       |  |
|  |  | Market   |  | Inference|  | Training |  | Data     |       |  |
|  |  | Module   |  | Module   |  | Module   |  | Module   |       |  |
|  |  | stocks,  |  | predict, |  | jobs,    |  | snapshots|       |  |
|  |  | OHLC,    |  | features,|  | logs,    |  | build,   |       |  |
|  |  | CRUD     |  | confid.  |  | status   |  | download |       |  |
|  |  +----------+  +----------+  +----------+  +----------+       |  |
|  |                      |                                        |  |
|  |              +-------+-------+                                |  |
|  |              | XGBoost Model |                                |  |
|  |              | (.ubj native) |                                |  |
|  |              | + meta.json   |                                |  |
|  |              | + encoder.pkl |                                |  |
|  |              +---------------+                                |  |
|  +-------------------------------------------------------------+  |
```

```
+-------------------------------+---------------------------------------+
                                |
                                v
+-----------------------------------------------------------------------+
|                           Data Layer                                  |
|  +------------------+    +------------------+    +------------------+ |
|  | PostgreSQL 16    |    | Apache Airflow   |    | DRAC HPC         | |
|  | market schema    |    | DAGs: seed,      |    | GPU Training     | |
|  | stocks + prices  |    | retrain, snapshot|    | Optuna HPO       | |
|  | user accounts    |    |                  |    | NVIDIA H100      | |
|  +------------------+    +------------------+    +------------------+ |
+-----------------------------------------------------------------------+
```

4.2 Technology Stack Summary

Layer	Technology	Version
Frontend (Web)	React, TypeScript, Vite, Tailwind CSS, TanStack Router	React 19, TS 5.x
Frontend (Data)	Streamlit	Latest
Backend Framework	FastAPI	0.115+
ORM	SQLAlchemy	2.0
Validation	Pydantic	v2
Database	PostgreSQL	16
ML Framework	XGBoost (native .ubj format)	2.x
HPO	Optuna	Latest
Feature Engineering	Custom Python module (53 indicators)	-
Pipeline	Apache Airflow	2.x
Reverse Proxy	Nginx	Latest
Containers	Docker, Docker Compose	Latest
CI/CD	GitHub Actions	-
Charts	TradingView Lightweight Charts	4.x
UI Components	shadcn/ui	Latest

5. Class Diagram

5.1 Backend Module Structure

```
backend/app/
+-- api/
|   +-- main.py                   # FastAPI router aggregation
|   +-- deps.py                   # Dependency injection (DB session)
|
+-- core/
|   +-- config.py                 # Settings (Pydantic BaseSettings)
|   +-- db.py                     # SQLAlchemy engine + session factory
|   +-- security.py               # Password hashing utilities
|
+-- modules/
    +-- market/
    |   +-- models.py             # Stock, DailyPrice (SQLAlchemy)
    |   +-- schemas.py            # StockRead, OHLCRead (Pydantic)
    |   +-- crud.py               # get_stock, get_ohlc, list_stocks
    |   +-- service.py            # MarketService (business logic)
    |   +-- api.py                # /market/stocks, /market/ohlc
    |
    +-- inference/
    |   +-- model_loader.py       # ModelManager (singleton, loads .ubj)
    |   +-- features.py           # prepare_features_for_prediction()
    |   +-- confidence.py         # calculate_confidence() [4 factors]
    |   +-- service.py            # InferenceService.predict_stock_price()
    |   +-- schemas.py            # PredictionResponse (Pydantic)
    |   +-- api.py                # /inference/predict/{symbol}
    |
    +-- training/
    |   +-- api.py                # /training/start, /training/status, /training/logs
    |
    +-- data/
        +-- api.py                # /data/snapshots (build, list, download)
```

5.2 Key Classes

```
+-------------------------+       +-------------------------+
| ModelManager (Singleton)|       | InferenceService        |
```

```
+-------------------------+       +-------------------------+
| - model: XGBRegressor   |       | + predict_stock_price() |
| - ticker_encoder: LE    |       |   - validate stock      |
| - feature_names: list   |       |   - fetch OHLC          |
| - metadata: dict        |       |   - calc 53 features    |
| + load_model()          |       |   - encode ticker       |
| + get_model()           |       |   - model.predict()     |
+-------------------------+       |   - calc confidence     |
                                  |   - return response     |
+-------------------------+       +-------------------------+
| MarketService           |
+-------------------------+       +-------------------------+
| + list_stocks(filters)  |       | ConfidenceCalculator    |
| + get_stock(symbol)     |       +-------------------------+
| + get_ohlc(symbol,days) |       | + volatility_score(35%) |
+-------------------------+       | + return_mag_score(30%) |
                                  | + volume_score(20%)     |
+-------------------------+       | + rsi_score(15%)        |
| Stock (SQLAlchemy)      |       | + calculate() -> 0.0-1.0|
+-------------------------+       +-------------------------+
| symbol: str (PK)        |
| name: str               |       +-------------------------+
| sector: str             |       | DailyPrice (SQLAlchemy) |
| industry: str           |       +-------------------------+
| exchange: str           |       | id: int (PK)            |
| ipo_date: date          |       | stock_symbol: str (FK)  |
| market_cap: float       |       | date: datetime          |
+-------------------------+       | open, high, low, close  |
                                  | volume: int             |
                                  +-------------------------+
```

6. Sequence Diagrams

6.1 Get Stock Prediction

```
User        Frontend        Nginx       Backend         DB          Model
 |              |              |            |             |            |
 |--[select]--->|              |            |             |            |
 |              |--GET /predict/AAPL------->|             |            |
 |              |              |            |--get_stock->|            |
 |              |              |            |<--Stock-----|            |
 |              |              |            |--get_ohlc-->|            |
 |              |              |            |<--OHLC[]----|            |
 |              |              |            |                          |
 |              |              |            |--calc_features(53)------>|
 |              |              |            |--encode_ticker---------->|
 |              |              |            |--model.predict()-------->|
 |              |              |            |<--predicted_return-------|
 |              |              |            |                          |
 |              |              |            |--calc_confidence()       |
 |              |              |            |                          |
 |              |<--PredictionResponse------|                          |
 |<--[card]----|              |            |                          |
```

6.2 View Stock Chart

```
User        Frontend        Nginx       Backend         DB
 |              |              |            |             |
 |--[click]---->|              |            |             |
 |              |--GET /stocks/AAPL-------->|             |
 |              |              |            |--get_stock->|
 |              |              |            |<--Stock-----|
 |              |--GET /ohlc?days=365------>|             |
 |              |              |            |--get_ohlc-->|
 |              |              |            |<--OHLC[]----|
 |              |<--JSON data--------------|             |
 |<--[chart]----|              |            |             |
 |              |              |            |             |
 |--[toggle prediction]------->|            |             |
 |              |--GET /predict/AAPL------->|             |
```

```
 |              |<--PredictionResponse------|             |
 |<--[overlay]--|              |            |             |
```

6.3 Admin: Add User

```
Admin       Frontend        Nginx       Backend         DB
 |              |             |            |             |
 |--[Add User]->|             |            |             |
 |<--[dialog]---|             |            |             |
 |--[fill form]>|             |            |             |
 |--[Save]----->|             |            |             |
 |              |--POST /users------------>|             |
 |              |             |            |--hash_pw--->|
 |              |             |            |--insert---->|
 |              |             |            |<--User------|
 |              |<--201 Created------------|             |
 |<--[updated]--|             |            |             |
```

Code

7. Project Source Code Description

7.1 Backend Modules

Module	Path	Files	Description
API Layer	`backend/app/api/`	main.py, deps.py	Router aggregation, DB session dependency injection
Market Module	`backend/app/modules/market/`	models.py, schemas.py, crud.py, service.py, api.py	Stock metadata, OHLC data CRUD, market API endpoints
Inference Module	`backend/app/modules/inference/`	model_loader.py, features.py, confidence.py, service.py, schemas.py, api.py	Model loading (native .ubj), 53-feature engineering, confidence scoring, prediction API
Training Module	`backend/app/modules/training/`	api.py	Training job management: start, status, log streaming
Data Module	`backend/app/modules/data/`	api.py	Dataset snapshot API: build, list, download
Core	`backend/app/core/`	config.py, db.py, security.py	Pydantic settings, SQLAlchemy engine, password hashing

7.2 Frontend Modules (React)

Module	Path	Description
Routes	`frontend/src/routes/`	TanStack Router file-based pages
- Dashboard	`routes/dashboard/index.tsx`	Summary cards with stock/sector counts
- Stocks List	`routes/dashboard/stocks.tsx`	Searchable stock table with sector grouping
- Stock Detail	`routes/dashboard/stocks.$symbol.tsx`	Charts, prediction overlay, stock metadata
- Predictions	`routes/dashboard/predictions.tsx`	Stock selector, predict, batch predict, cards
- Training	`routes/dashboard/training.tsx`	Training monitor with log streaming
- Settings	`routes/dashboard/settings.tsx`	Profile, password, account deletion
- Admin	`routes/dashboard/admin.tsx`	User management (superuser only)
Components	`frontend/src/components/`	Reusable UI components
- Charts	`components/Charts/StockChart.tsx`	TradingView Lightweight Charts wrapper
- Sidebar	`components/Sidebar/AppSidebar.tsx`	Navigation sidebar with theme switcher

- Admin	`components/Admin/`	AddUser, EditUser, DeleteUser dialogs
Services	`frontend/src/services/`	API client functions
- inference.ts		predictStock(), predict()
- market.ts		getStocks(), getOHLC()
Hooks	`frontend/src/hooks/`	React Query hooks for data fetching
Types	`frontend/src/types/`	TypeScript interfaces matching backend schemas

7.3 ML Pipeline

Module	Path	Description
Data Prep	`ml/scripts/data_prep/`	FMP download, data processing, hosting scripts
Training	`ml/scripts/training/`	XGBoost training with Optuna HPO
Inference	`ml/scripts/inference/`	Test inference script
Gap Filling	`ml/scripts/fill_gap.py`	Fill 5-min OHLCV data gaps (2020-2023)
FMP Import	`ml/scripts/import_fmp_to_db.py`	Load FMP data into PostgreSQL
Notebooks	`ml/notebooks/`	XG_BOOST_1, XG_BOOST_2, test_global_model
Model Artifacts	`ml/models/`	Native .ubj model + meta.json + encoder

7.4 Infrastructure

File	Description
`docker-compose.yml`	All service definitions (backend, frontend, DB, Nginx, Airflow, Streamlit)
`docker/nginx/nginx.conf`	Nginx reverse proxy configuration with SSL
`airflow/dags/`	DAGs for data seeding, model retraining, snapshots
`.github/workflows/ci.yml`	CI pipeline (6 jobs)
`docs_server_setup.md`	DRI production server setup documentation
`scripts/sync_dw_to_local_postgres.py`	Remote-to-local DB sync with SSH tunnel

8. Testing Code Description

8.1 Test Structure

```
backend/tests/
+-- conftest.py                    # Shared DB session, TestClient fixtures
+-- api/
|   +-- test_utils.py             # Health check endpoint test
+-- modules/
    +-- conftest.py               # Market test data: stocks + OHLC seeding
    +-- test_market.py            # Integration: market API endpoints
    +-- test_market_crud.py       # Unit: CRUD operations
    +-- test_market_service.py    # Unit: MarketService business logic
    +-- test_inference.py         # Integration: inference API (mocked model)
    +-- test_confidence.py        # Unit: confidence scoring functions
```

8.2 Test Inventory

Test File	Type	Tests	Description
`test_utils.py`	Unit	1	Health check returns true with 200 status
`test_market.py`	Integration	4	GET /stocks, GET /stocks/{symbol}, GET /ohlc, 404 handling
`test_market_crud.py`	Unit	6	Create/read stocks, create/read prices, filtering, edge cases
`test_market_service.py`	Unit	3	list_stocks, get_stock, get_ohlc via service layer
`test_inference.py`	Integration	3	Prediction endpoint, 404 for invalid stock, response schema
`test_confidence.py`	Unit	8	Volatility score, RSI score, volume score, return magnitude, composite, edge cases (NaN, zero, extreme values)
Total		**25**	

8.3 Test Fixtures

Fixture	Location	Description
`db_session`	`conftest.py`	In-memory SQLite session for isolated testing
`client`	`conftest.py`	FastAPI TestClient with overridden DB dependency
`seed_market_data`	`modules/conftest.py`	Creates 4 stocks + 5 days of OHLC data per stock

9. Continuous Integration Report

9.1 Pipeline Configuration

File: `.github/workflows/ci.yml` **Trigger:** Push or Pull Request to `main` branch **Runner:** Ubuntu Latest (GitHub-hosted)

9.2 Job Summary

Job	Steps	Duration	Status
backend-lint	Install deps -> Ruff check	~30s	Passing
backend-typecheck	Install deps -> Mypy strict	~45s	Passing
backend-test	Install deps -> Pytest with coverage	~2 min	Passing
frontend-lint	Install deps -> Biome check	~20s	Passing
frontend-build	Install deps -> Vite production build	~45s	Passing
docker-build	Docker Compose build all services	~3 min	Passing

Total Pipeline Time: ~7 minutes

9.3 Test Results (Latest Run)

```
========= test session starts =========
collected 25 items

tests/api/test_utils.py          .         [  4%]
tests/modules/test_market.py     ....      [ 20%]
tests/modules/test_market_crud.py ......   [ 44%]
tests/modules/test_market_service.py ...   [ 56%]
tests/modules/test_inference.py  ...       [ 68%]
tests/modules/test_confidence.py ........  [100%]

========= 25 passed in 4.2s =========
```

9.4 Code Quality Metrics

Metric	Tool	Result

Python linting	Ruff	0 errors, 0 warnings
Python types	Mypy (strict)	0 errors
TypeScript linting	Biome	0 errors, 0 warnings
Frontend build	Vite	Successful, 0 warnings
Docker build	Docker Compose	All images build successfully
Test coverage	Pytest-cov	~75% line coverage